

Daily Scholar Papers Report — 2026-06-06

2026-06-06

Daily Scholar Papers Report — 2026-06-06

[Download PDF](#)

Window covered: 2026-06-05 → 2026-06-06 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

A thin three-alert window with no fresh Outstanding hit. Two of three alerts are deferred at Stage-1: one is off-topic for the program-analysis / vulnerability-detection focus, and the other is a Paderborn-hosted re-listing of the same Jaralyzer / bytecode-centric Java SCA work that shipped as yesterday's [Outstanding](#) — same author, near-identical abstract opening, no signal that the document is a meaningful superset. That leaves the day's only Keep, surfaced via the followed-researcher override on Armando Solar-Lezama's Scholar feed: a 2016 NeurIPS paper on *sampling* from posteriors over typed program sketches, re-indexed today via MIT DSpace. It is well outside the daily vuln/SCA cluster but worth a short surfacing because it is the constraint-based-sampling kernel that Ellis went on to develop into DreamCoder / library-learning, and because the *replace-an-NP-complete-exact-match-with-a-tractable-approximation* move is the same one Jaralyzer makes on subgraph isomorphism — a primitive worth keeping in the toolkit. No user-curated self-emails arrived.

Outstanding: 0 · **Keep:** 1 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
2.1	Sampling for Bayesian Program Learning	Ellis, Solar-Lezama, Tenenbaum	NeurIPS 2016 (re-indexed 2026-06-05)	NeurIPS

1. Outstanding

No Outstanding paper today. The day's highest-fit candidate (a Paderborn-hosted writeup of the Schott / Jaralyzer line of work) duplicates yesterday's [Outstanding deep-read](#) of the ICSE 2026 / arXiv 2510.19393 conference version of the same work — same author, byte-identical abstract opening — and is not re-treated here.

2. Keep

2.1 · PROGRAM SYNTHESIS · NeurIPS 2016 re-indexed via MIT DSpace — replaces exact posterior sampling over typed program sketches with random-parity-constraint hashing, provably arbitrarily close to the true posterior on 22 string-edit / programming-by-example tasks

2.1 [Sampling for Bayesian Program Learning](#) — Ellis, Solar-Lezama, Tenenbaum (MIT BCS + CSAIL), NeurIPS 2016 (alert hit 2026-06-05 via MIT DSpace re-index)

Authors: Kevin Ellis (MIT BCS, now Cornell CS), Armando Solar-Lezama (MIT CSAIL), Joshua B. Tenenbaum (MIT BCS) **Venue:** Advances in Neural Information Processing Systems 29 (NeurIPS 2016) **Mirrors:** [NeurIPS proceedings](#) · [PDF \(MIT DSpace\)](#) · [DSpace record](#) · [Semantic Scholar](#)

Why surfaced today. Followed-researcher track (Solar-Lezama). The 2026-06-05 alert is a Scholar re-indexing of the DSpace mirror, not a new paper; the writeup below is intentionally brief.

Problem. Learning programs from a handful of input-output examples (the FlashFill-style programming-by-example regime) typically returns a single best program, which is brittle when only one or two examples are provided and many semantically distinct programs fit them. The paper reframes this as Bayesian inference over a typed program DSL and asks for *samples* from the posterior conditioned on the data, so the downstream system (or the user) can see ambiguity directly.

Approach. Two combined ingredients.

1. *Solver-based program synthesis.* The DSL is encoded as a SKETCH-style typed sketch; the conditioning on input-output examples is a constraint system handed to an off-the-shelf SMT solver.
2. *Sampling via random parity constraints.* Exact posterior sampling over the program space is intractable. The paper composes random parity (XOR) constraints with the solver, in the spirit of Ermon-Selman-Gomes hashing for #SAT, to draw samples whose distribution is provably arbitrarily close to the true posterior. Quality of the approximation is controlled by the parity-constraint parameters and yields explicit closeness bounds.

The system is implemented as `PROGRAMSAMPLE`.

Evaluation. 22 program-induction benchmarks across two domains: string editing (FlashFill-style date / name munging) and computer-aided programming. Figure 1 shows the canonical example — input “1/21/2001” → “01” — for which `PROGRAMSAMPLE` returns four ranked program hypotheses (e.g. `substr(pos('0', -1), -1), const('01'), substr(-2, -1)`) so the ambiguity intrinsic to one-shot programming-by-example is preserved instead of silently resolved.

Why this matters in 2026. Three reasons to keep this on the radar.

1. The constraint-based-sampling kernel here is the direct precursor of the Ellis-line library-learning / DreamCoder work; it is also the move that the 2025 LLM-guided POMDP-induction paper (arXiv 2505.02216, surfaced by the same Scholar listing) reuses with an LLM proposer in front.
2. The *replace-an-NP-complete-exact-match-with-a-tractable-approximation* move parallels what yesterday’s Jaralyzer paper does on subgraph isomorphism (the Bowman-Huang triplet trick +

the $|NT \cap T_m| \geq |PT \cap T_m|$ decision rule). Two domains, identical move — worth keeping in the toolkit for any future syntactic / semantic-equivalence work.

3. For vuln-detection / SCA work the technique is most directly useful as a probabilistic-spec-mining primitive: sampling programs that explain observed traces, with calibrated uncertainty rather than a brittle MAP point estimate. That is a long-term connection, not a today-call.

License. NeurIPS proceedings (non-exclusive distribution to NeurIPS; author copyright retained). No CC license attached — no figure embed, brief prose only.

3. Borderline High-Priority

None today.

Cross-Paper Synthesis

Only one paper was deep-treated today, so no genuine cross-paper synthesis is possible. The closest connection worth flagging is *retrospective and across reports*: this Keep and yesterday's Outstanding (Schott et al., Jaralyzer) make the same architectural move at the inner-loop level — replace an NP-complete exact comparison (subgraph isomorphism in Jaralyzer; exact posterior sampling here) with a calibrated approximation that turns the problem into either constant-time set overlap or a polynomial sequence of solver calls under random-projection bounds. The Bowman-Huang triplet trick and the Ermon-Selman-Gomes parity-constraint hashing are different surface techniques for what is, at the algorithmic level, the same trade.

Writing & Rationale Insights

A thin window is a useful stress-test for the pipeline. Three points worth noting.

The first is on Stage-1 *saturation within the report window*. When two consecutive alerts on the same author and same topic land 24 h apart, the right move is to point back to the prior deep-read rather than re-summarise, unless the second document is a noticeable superset (longer evaluation, new contribution claim, distinct artefact). Today's Schott / Paderborn alert offered no such signal — the abstract opening was byte-identical to yesterday's analysed paper — so the saturation skip was clean.

The second is on *the followed-researcher override*. The override exists to keep visibility on tracked researchers' feeds, not to elevate every hit to Outstanding. A re-indexed 2016 NeurIPS paper is correctly surfaced as a brief Keep — the user sees the index hit on Solar-Lezama's feed, but the day's Outstanding budget is not spent on a ten-year-old paper.

The third is that *off-topic followed-researcher co-authorship is still off-topic*. The followed-researcher exemption only protects against the preference-demoted auto-skip; the normal Stage-1 topical-fit gate still applies. A TDSC outsourced-database integrity paper does not pull toward the user's focus area simply because a followed researcher appears in the author list — and the rubric handled this correctly today without losing signal.